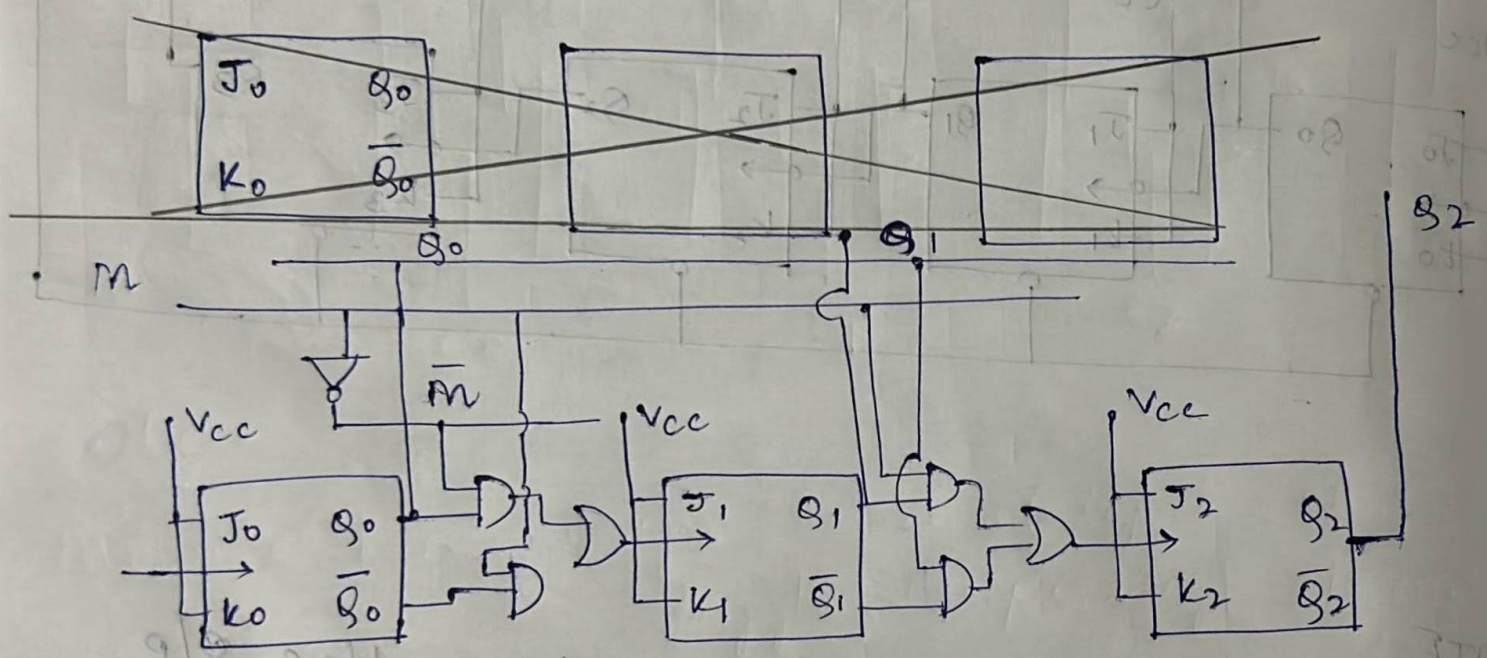
Q6.] Asynchronous UP/DOWN Counter (3-bit)

Ans:-



Can use both UP/DOWN characteristic table.

Q7] Generate 70% clock cycle (Quiz Question).

→ Total clock cycle,

$T_C = 10ns$ $\qquad T_{ON} + T_{OFF}$

```
initial
  begin
  clk = 0;
  always
  #3 clk = 1;
  #7 clk = 0;
  end
```

Q8] Generate clock of 40% duty cycle.

→
```
initial
  begin
  clk = 0;
  always
  #42 clk = 1;
  #28 clk = 0;
  end
```

$T_C = 70ns$

10 - 4

$70 - ? = \frac{70 \times 4}{100}$

$= 28ns \rightarrow ON$

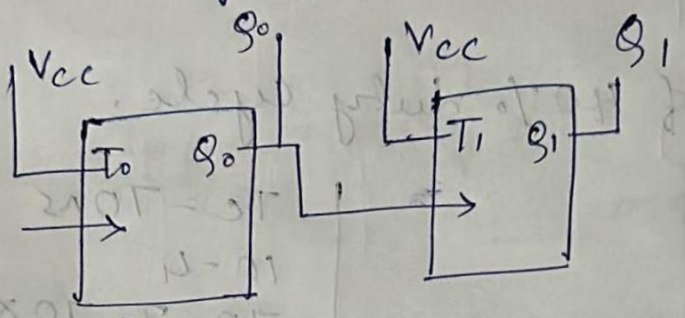
70
-28
42 → OFF

Q9] Write verilog code for Synchronous counter with counting sequence 4-7-2-1-3..4-7..

Ans:]
```
module C
  begin
   if (rst)
   count = 4'b0000;
  else
   case (count)
  4'b0100 : count = 4'b0111;
  4'b0111 : count = 4'b0010;
```

```
4'b0010 : count = 4'b0001;
4'b0001 : count = 4'b0011;
4'b0011 : count = 4'b0100;
default : count = 4'b0100;
  endcase
  end
endmodule.
```

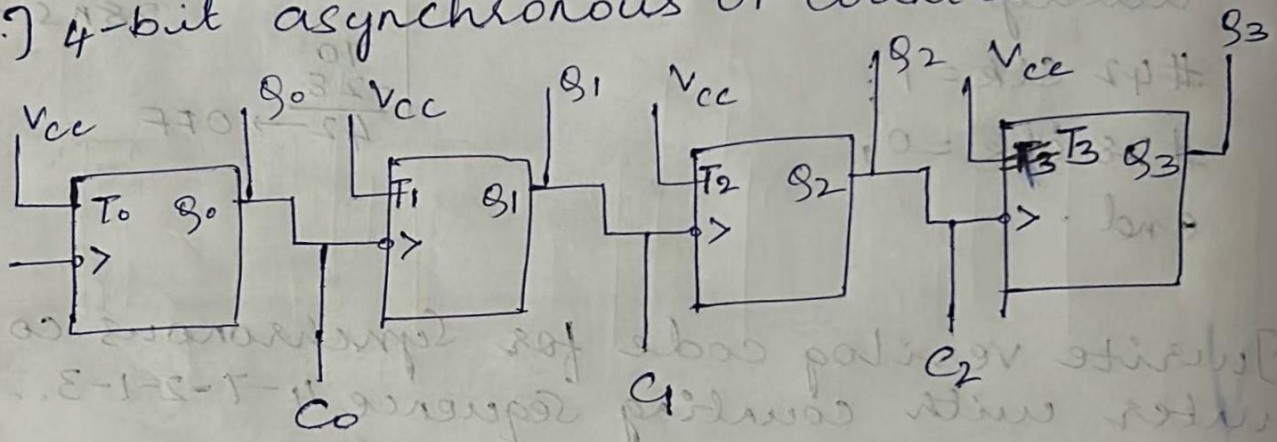
Q] 2 bit asynchronous up counter.



NOTE

mod-16 (0-15) } Binary counter

Q11] 4-bit asynchronous UP counter.



code:-

```
Module bin_asyn_counter(input clk, rst,
                        toggle, Output reg[3:0] Count);

  wire c0, c1, c2;
  assign c0 = count[0], c1 = Count[1], c2 = count[2];
  always @(negedge rst, negedge clk)
    if (rst).
    count[0] <= 1'b0;
```

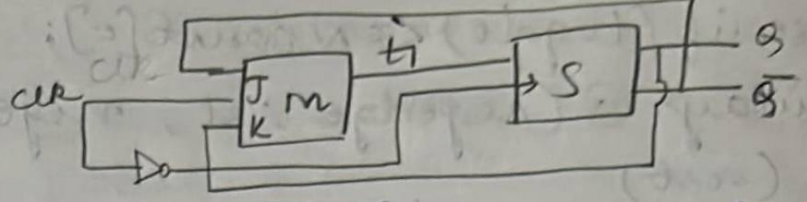
```
else if (toggle) <= ~count[0];
always @(negedge rst, negedge C0)
if (rst)
  count[1] <= 1'b0;
  else if (toggle)
    count[1] <= ~count[1];
always @(negedge rst, negedge C1)
  if (rst)
    count[2] <= 1'b0;
  else if (toggle)
    count[2] <= ~count[2];
always @(negedge rst, negedge C2)
  if (rst)
    count[3] <= 1'b0;
  else if (toggle)
    count[3] <= ~count[3];
```

Q12] Master-Slave JK flip flop.

Ans:-

```
module jkff (input j, k, clk, output reg Q);
  always @(clk)
  begin
    case ({j, k})
    2'b00 : Q <= Q;
    2'b01 : Q <= 0;
    2'b10 : Q <= 1;
    2'b11 : Q <= ~Q;
    default: Q <= z;
```

```
endcase
end
endmodule.
```



→ module jkms(input j, k, clk, output q);

```
  wire t1
  jkff master (j, k, ~~d~~clk, t1);
  jkff slave (t1, ~t1, ~clk, q);
  end module
```